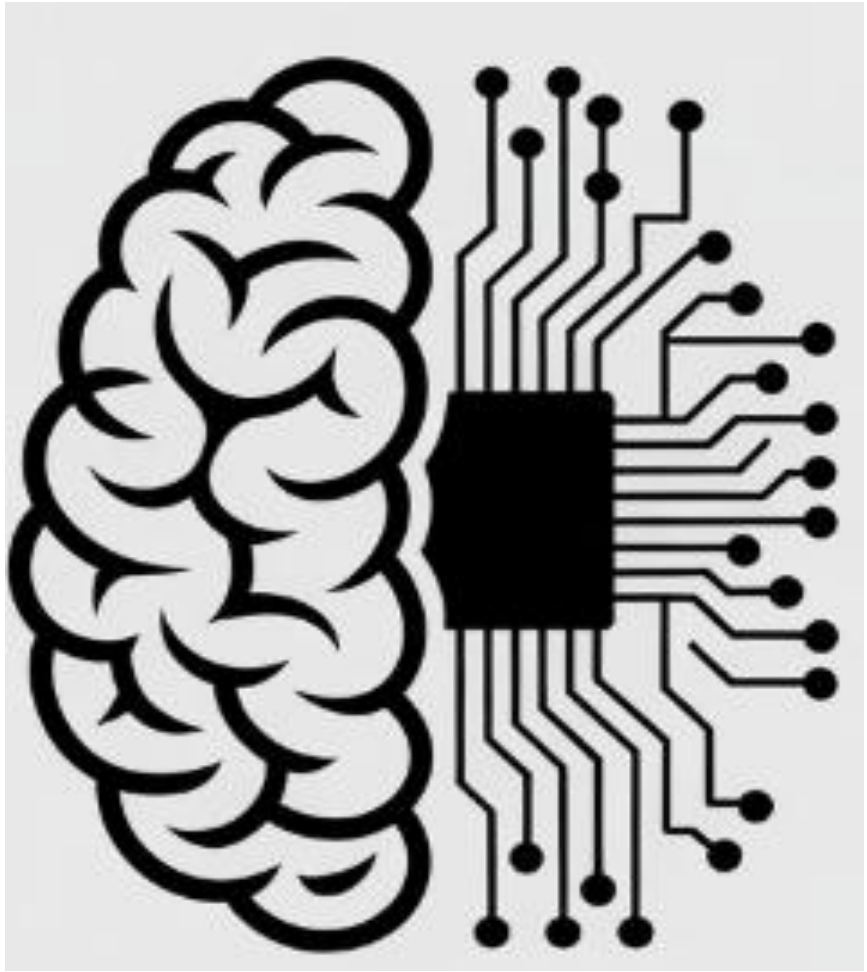


Convolution Neural Network (CNNs)

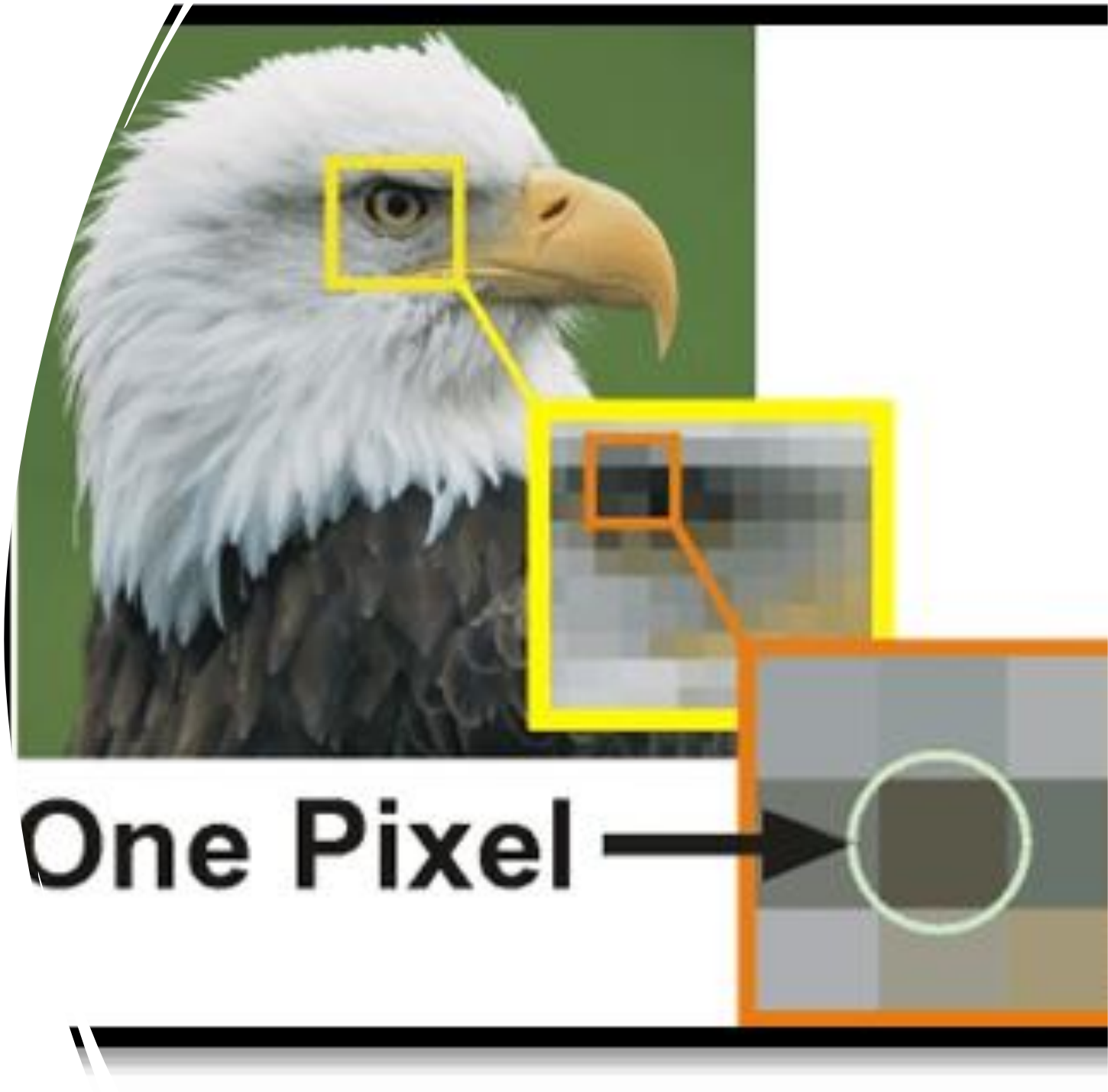
Dr. Sobia Tariq Javed

CNNs

- CNNs are a class of neural networks designed to process grid-like data, especially **images**.
- Automatically learn **spatial features (edges, textures, shapes)** using **convolution operations**.

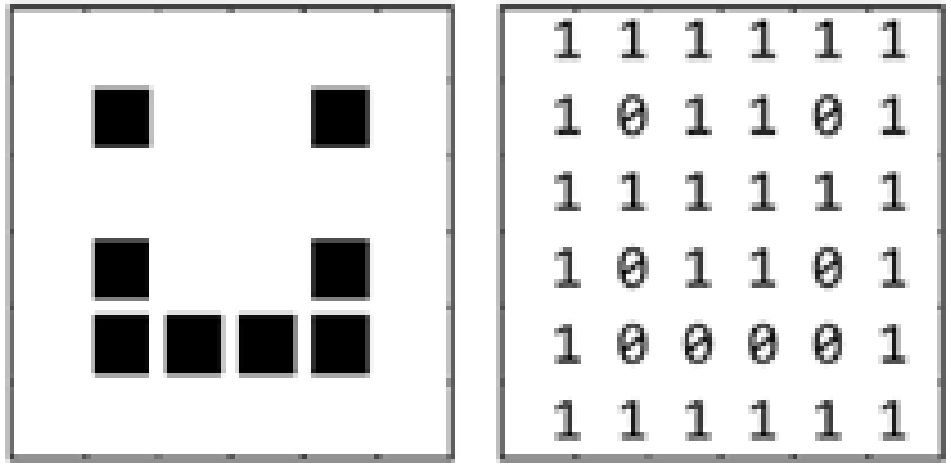
Digital Image

- It is an image stored in a computer as **a grid of tiny units** called **Pixel**, where each pixel contains **color or intensity information**.



Types of images

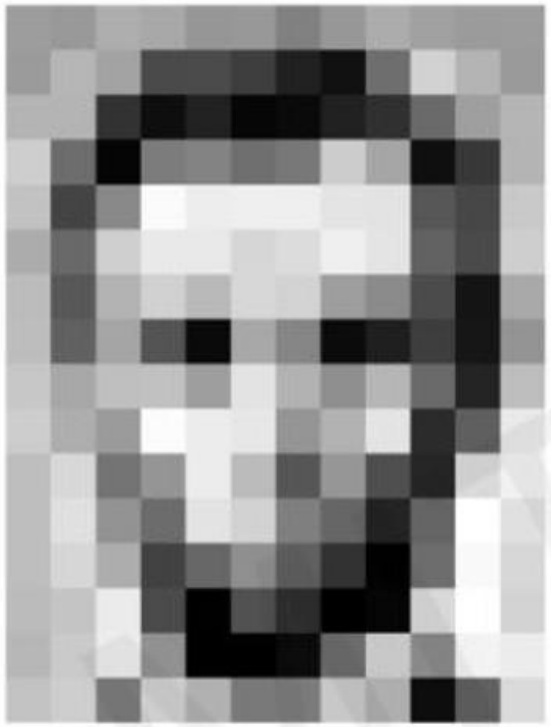
- **Binary Image**
  - Only two values: black and white (0 or 1)



1	1	1	1	1	1
1	0	1	1	0	1
1	1	1	1	1	1
1	0	1	1	0	1
1	0	0	0	0	1
1	1	1	1	1	1

Types of images

- **Grayscale Image**
  - Shades of gray (0 = black, 255 = white)



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Types of images

- **Color Image (RGB)**
  - Uses three channels: Red, Green, Blue to represent colors

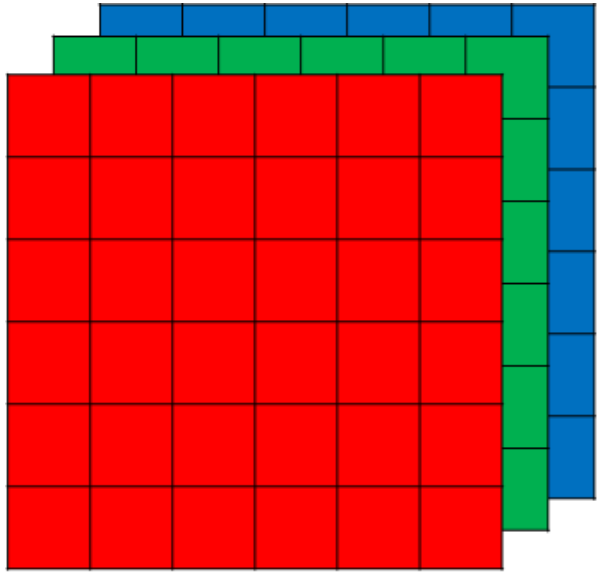
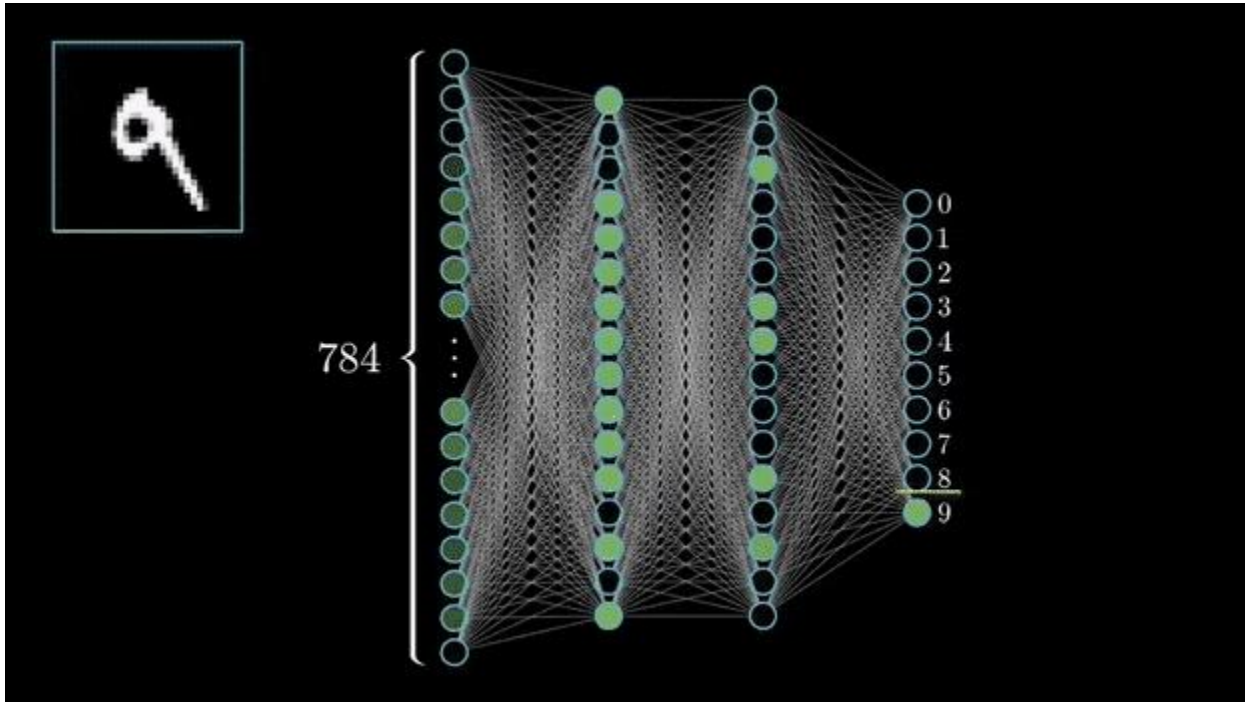


Image Classification through ANN

- In a classification task with digits from 0 to 9, the output layer would have 10 neurons.
- Image = 28x28
- Input layer has 784 neurons



Problems with ANN with image

- High Computational Cost
- Overfitting
- Large architecture → large trainable parameters
- Loss of important information like spatial arrangement of pixel

Convolution Operation

- It is a **mathematical operation** where a **filter (kernel)** slides over **input data**, **multiplying and summing** overlapping values to produce a new output (feature map).
- Image Size: n x n
- Filter size: f x f

$$\lfloor n - f + 1 \rfloor \quad \times \quad \lfloor n - f + 1 \rfloor$$

Convolution

- The mask over the upper-left corner of the image.
- 0(0) +1/2(0) +0(0) +1/2(0) +1(3) +1/2(0) + 0(0) +1/2(0) +0(0) =3

0	0	0	0	0	0	0
0	3	0	5	0	7	0
0	0	0	0	0	0	0
0	2	0	7	0	6	0
0	0	0	0	0	0	0
0	3	0	4	0	9	0
0	0	0	0	0	0	0

*

0	1/2	0
1/2	1	½
0	1/2	0

=

3				

Convolution

- The mask over the upper-left corner of the image.
- 0(0) +1/2(0) +0(0) +1/2(3) +1(0) +1/2(5) + 0(0) +1/2(0) +0(0) =4

0	0	0	0	0	0	0
0	3	0	5	0	7	0
0	0	0	0	0	0	0
0	2	0	7	0	6	0
0	0	0	0	0	0	0
0	3	0	4	0	9	0
0	0	0	0	0	0	0

*

0	1/2	0
1/2	1	½
0	1/2	0

=

3	4			

Convolution

- The mask over the upper-left corner of the image.
- 0(0) +1/2(0) +0(0) +1/2(0) +1(5) +1/2(0) + 0(0) +1/2(0) +0(0) =5

0	0	0	0	0	0	0
0	3	0	5	0	7	0
0	0	0	0	0	0	0
0	2	0	7	0	6	0
0	0	0	0	0	0	0
0	3	0	4	0	9	0
0	0	0	0	0	0	0

*

0	1/2	0
1/2	1	½
0	1/2	0

=

3	4	5		

Convolution

- The mask over the upper-left corner of the image.
- 0(0) +1/2(0) +0(0) +1/2(5) +1(0) +1/2(7) + 0(0) +1/2(0) +0(0) =6

0	0	0	0	0	0	0
0	3	0	5	0	7	0
0	0	0	0	0	0	0
0	2	0	7	0	6	0
0	0	0	0	0	0	0
0	3	0	4	0	9	0
0	0	0	0	0	0	0

*

0	1/2	0
1/2	1	½
0	1/2	0

=

3	4	5	6	

Convolution

- The mask over the upper-left corner of the image.
- 0(0) +1/2(0) +0(0) +1/2(0) +1(7) +1/2(0) + 0(0) +1/2(0) +0(0) =7

0	0	0	0	0	0	0
0	3	0	5	0	7	0
0	0	0	0	0	0	0
0	2	0	7	0	6	0
0	0	0	0	0	0	0
0	3	0	4	0	9	0
0	0	0	0	0	0	0

*

0	1/2	0
1/2	1	½
0	1/2	0

=

3	4	5	6	7

Convolution

- The mask over the upper-left corner of the image.
- 0(0) +1/2(3) +0(0) +1/2(0) +1(0) +1/2(0) + 0(0) +1/2(2) +0(0) =2.5

0	0	0	0	0	0	0
0	3	0	5	0	7	0
0	0	0	0	0	0	0
0	2	0	7	0	6	0
0	0	0	0	0	0	0
0	3	0	4	0	9	0
0	0	0	0	0	0	0

*

0	1/2	0
1/2	1	½
0	1/2	0

=

3	4	5	6	7
2.5				

Convolution Operation

- **Kernel size:** Usually much smaller than the input (e.g., 3×3 or 5×5).
- **Purpose:** Detects specific features such as edges, textures, or patterns.
- The kernel works as a "**feature detector**" that scans the input and highlights what it’s designed to find.

Types of Kernel

- Edge Detection-Vertical

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

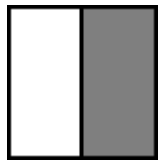
1	0	-1
1	0	-1
1	0	-1

=

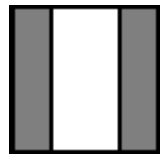
0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

1	0	-1
1	0	-1
1	0	-1

Vertical



*



Types of Kernel

- Edge Detection-Horizontal

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

$*$

1	1	1
0	0	0
-1	-1	-1

$=$

0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

1	1	1
0	0	0
-1	-1	-1

Horizontal

Types of Kernel

- Smoothing/Blur

$$\frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$$

- Sharpening Kernels

$$\begin{bmatrix}0 & -1 & 0\\ -1 & 5 & -1\\ 0 & -1 & 0\end{bmatrix}$$

- Emboss Kernels

$$\begin{bmatrix}-2 & -1 & 0\\ -1 & 1 & 1\\ 0 & 1 & 2\end{bmatrix}$$

Convolution process

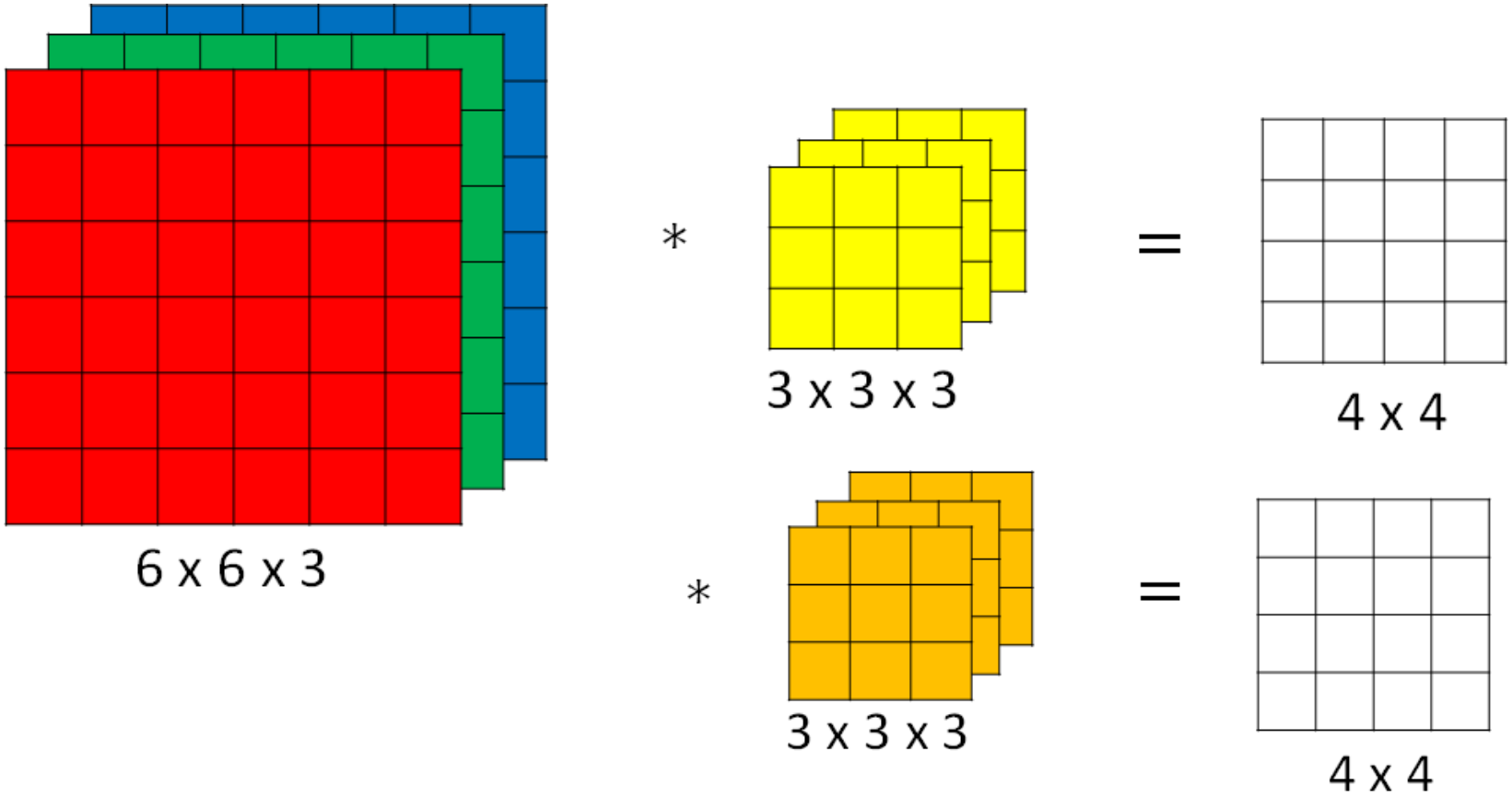


Image Size: **n x n**
Filter size: **f x f**

$$\left\lfloor \frac{n-f}{s} + 1 \right\rfloor \quad \times \quad \left\lfloor \frac{n-f}{s} + 1 \right\rfloor$$

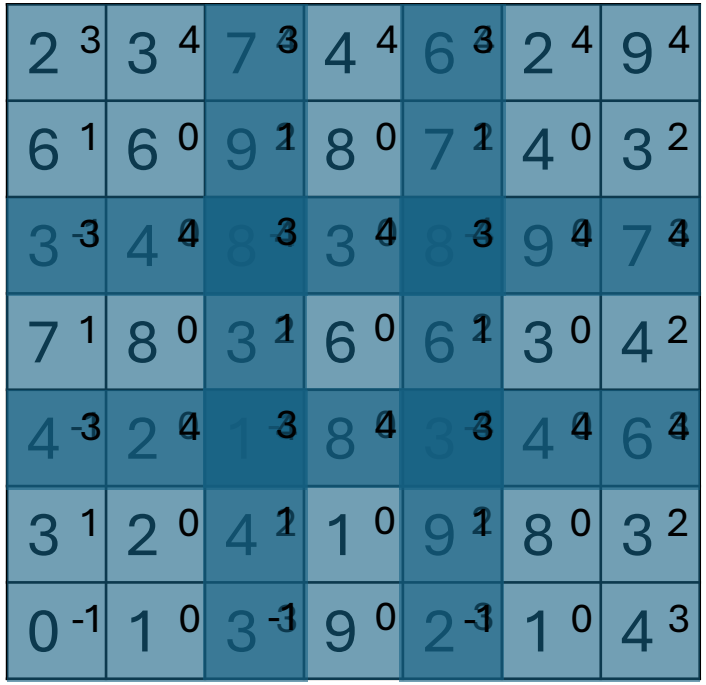
Stride

- **Stride** → how much the filter moves

- **Stride = 1** → moves one pixel at a time (more detailed output)
- **Stride = 2** (or higher) → skips pixels (smaller output, faster computation)

- **Larger stride** → reduces output size (downsampling) → coarse features
- **Smaller stride** → preserves more spatial detail → fine-grained features

Stride 2, n=7x7, filter=3x3



*

3	4	4
1	0	2
-1	0	3

=

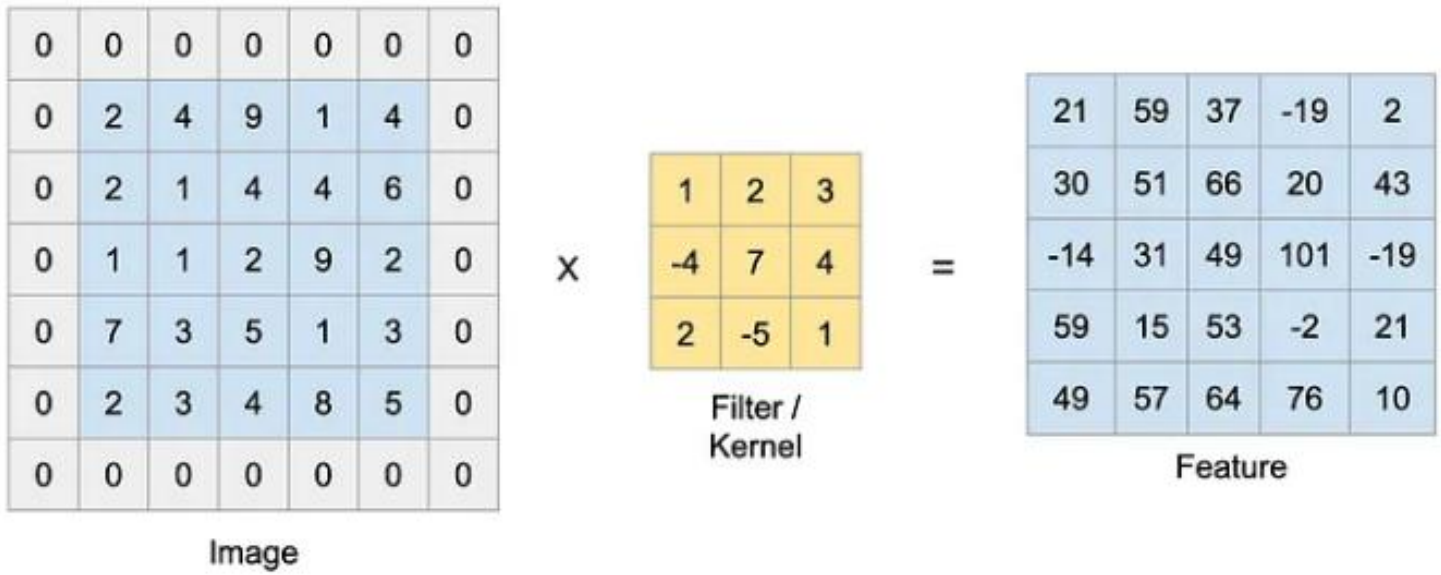
91	**100**	**83**
69	91	127
44	72	74

Padding

- **Padding** → controls output size
- Prevents **loss of spatial information** at edges
- Helps control **output size**
- Allows deeper networks without **rapid shrinking**

- **Types of Padding**
  - **Valid padding** → no padding (output shrinks)
  - **Same padding** → adds zeros so output size remains the same

Padding

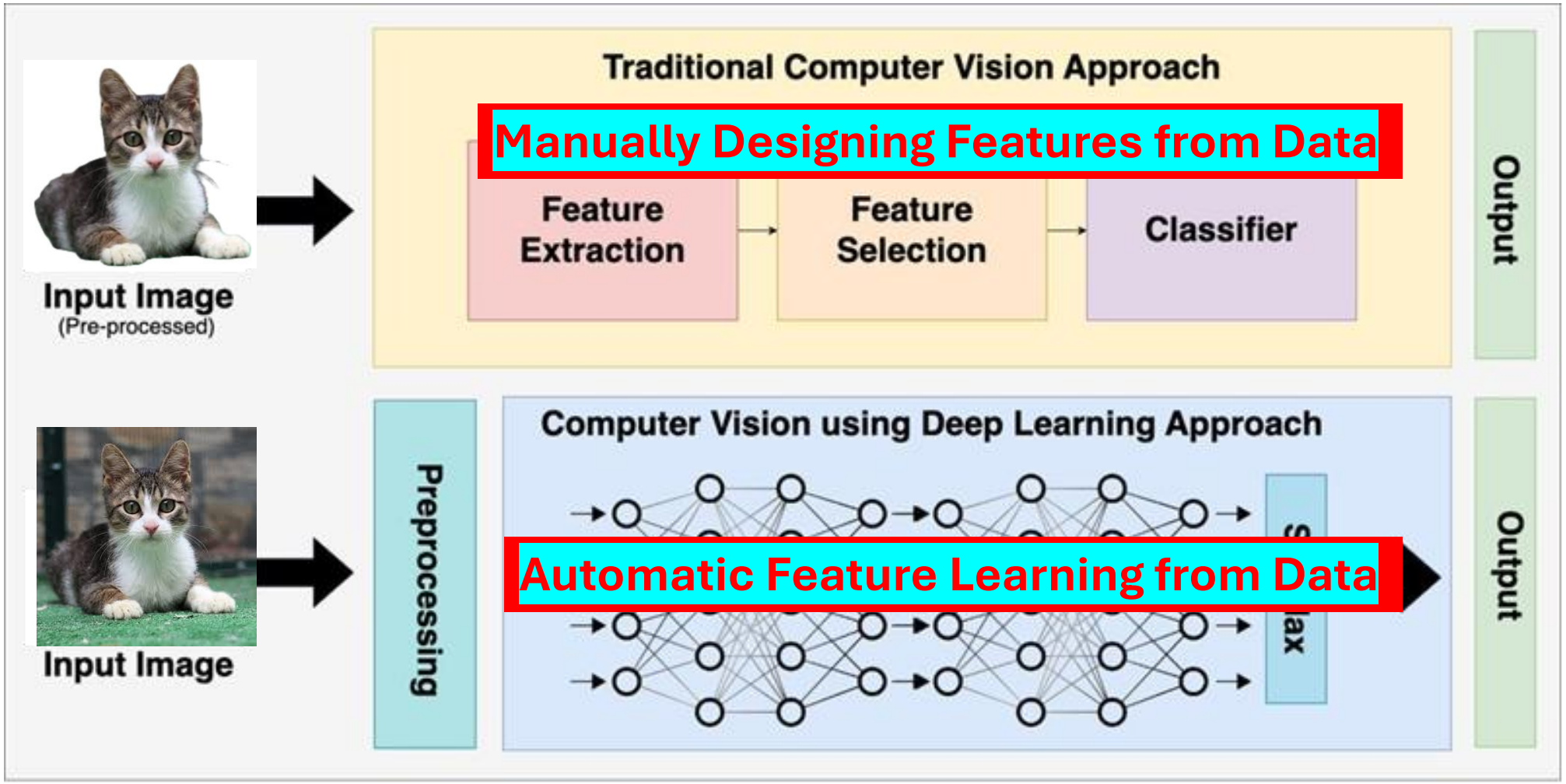

0 0 0 0 0 0 0
0 2 4 9 1 4 0
0 2 1 4 4 6 0
0 1 1 2 9 2 0
0 7 3 5 1 3 0
0 2 3 4 8 5 0
0 0 0 0 0 0 0
Image
X
1 2 3
-4 7 4
2 -5 1
Filter / Kernel
=
21 59 37 -19 2
30 51 66 20 43
-14 31 49 101 -19
59 15 53 -2 21
49 57 64 76 10
Feature

Feature Map Size

- Image Size: n x n
- Filter size: f x f
- Padding: p
- Stride: s

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

Traditional Computer Vision vs. Deep Learning

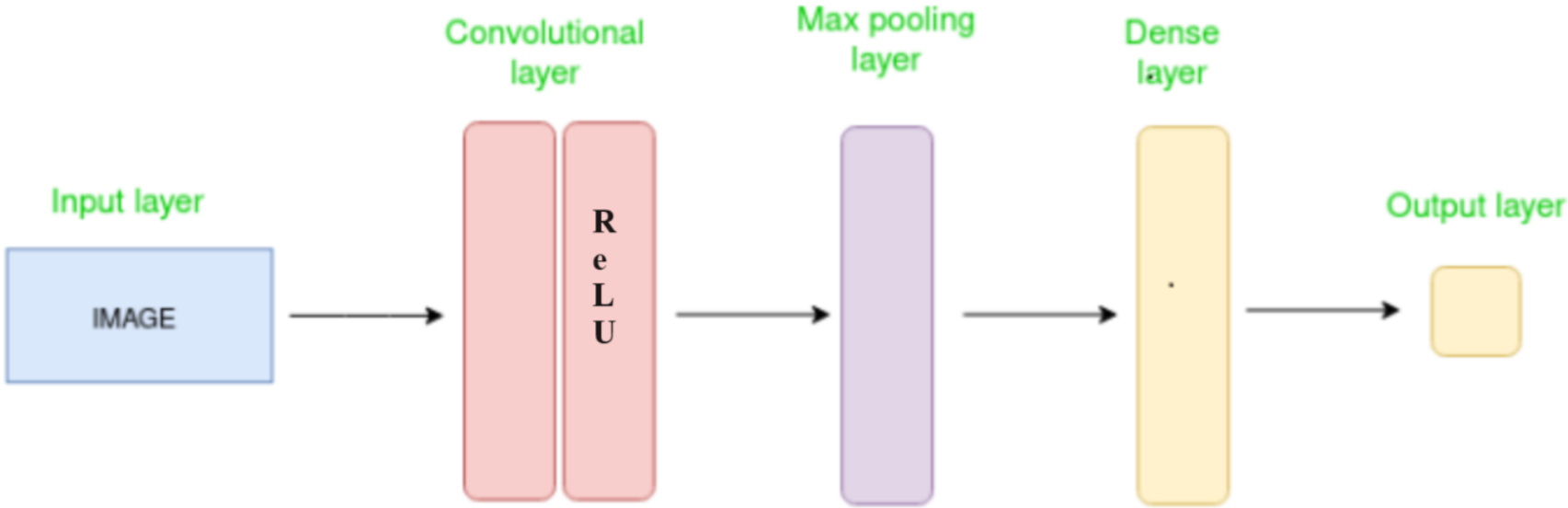


CNN

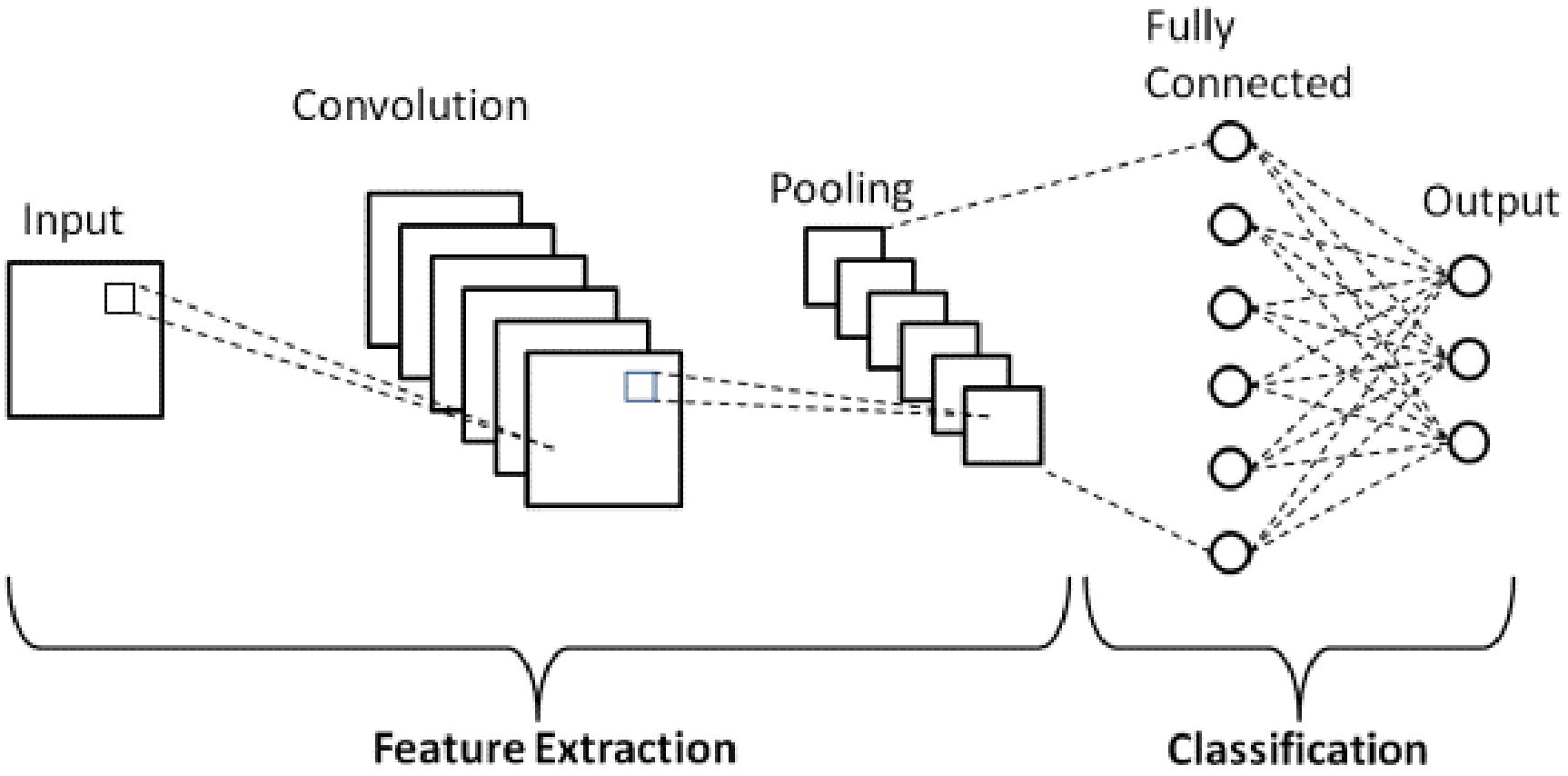
- A Convolutional Neural Networks (CNN) is a deep learning architecture that learns features directly from raw data.
  - Particularly effective for **image analysis** and **object recognition**
  - Automatically detects patterns such as **edges, textures, and shapes**
  - Also applicable to non-image data (e.g., **audio, time series, signals**)

CNN

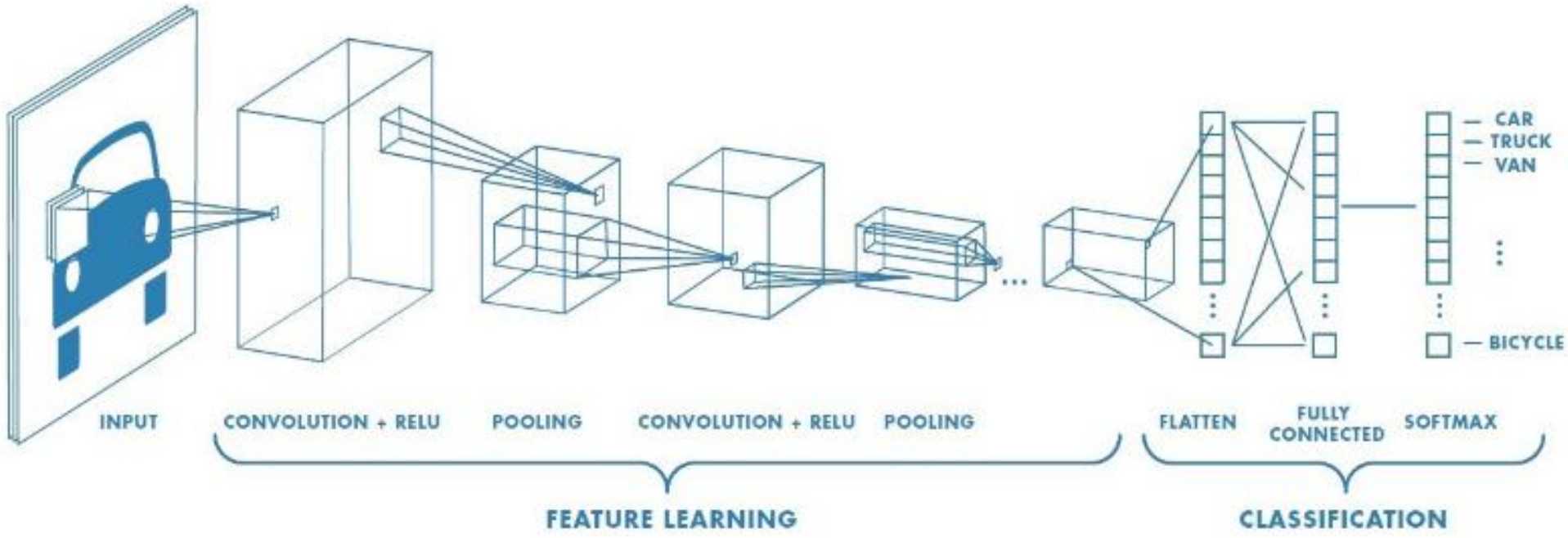
- Types of Layers
  - Input Layer
  - Convolution Layer
  - Pooling Layer
  - Fully connected Layer
  - Output Layer



CNN


Fully
Connected
Convolution
Pooling
Output
Input
Feature Extraction
Classification

CNN Architecture



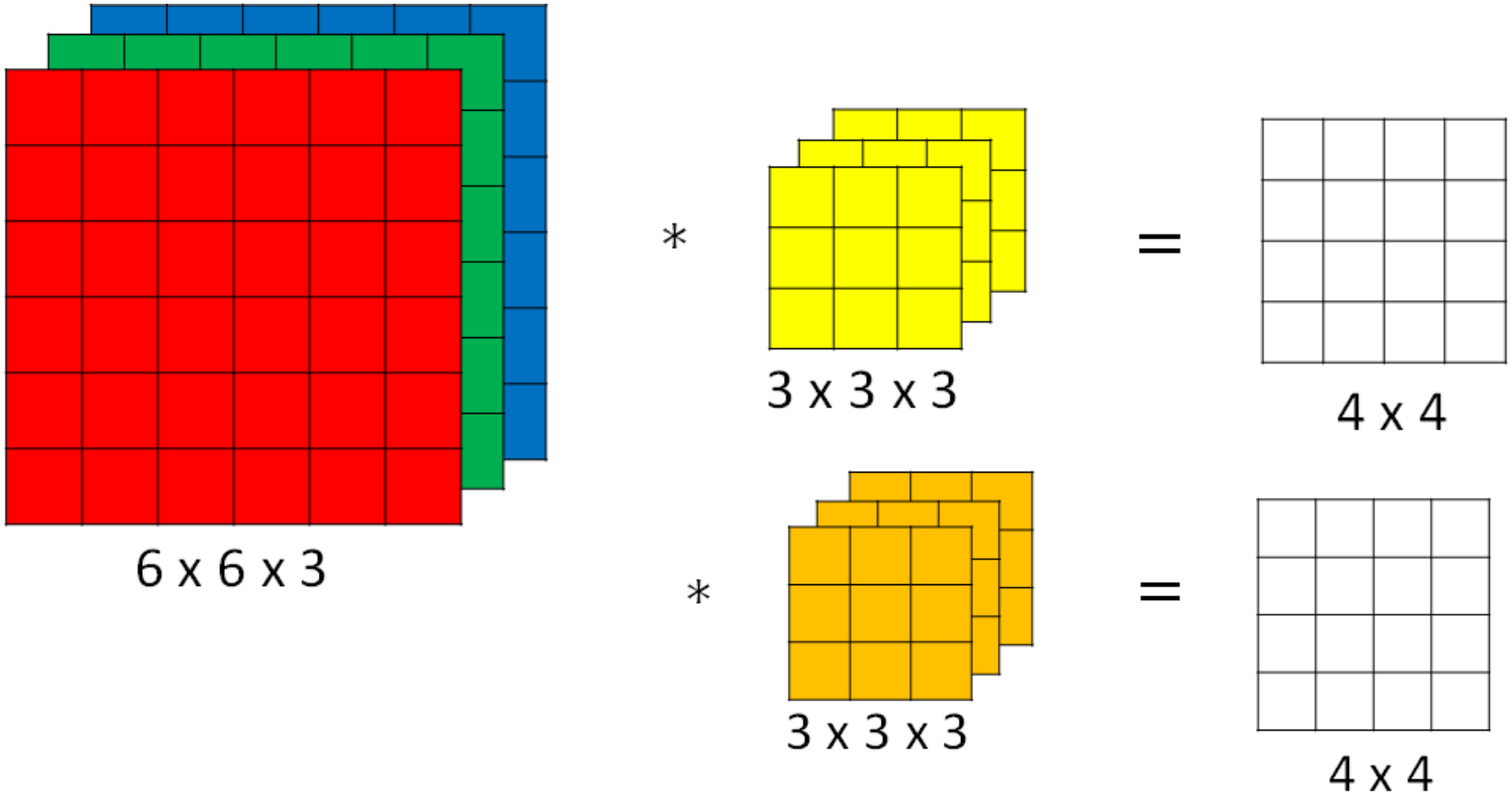
Convolution Layer

- It is the core building block of a CNNs, responsible for **extracting features** from input data (e.g., **images**) through convolution operation using **learnable filters.**

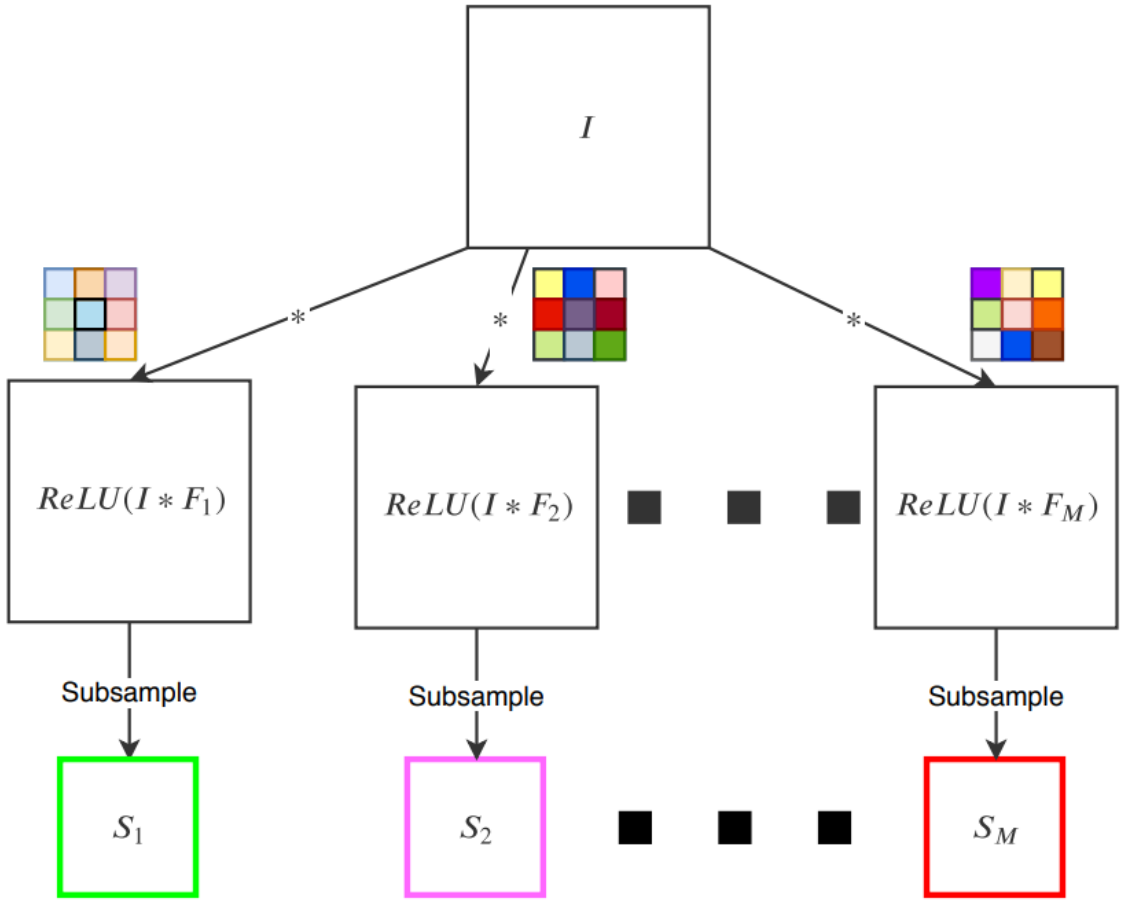
- After convolution, an activation function is applied to introduce non-linearity—most commonly ReLU

- Instead of **manually designing features**, the convolution layer automatically **learns relevant patterns directly from data**, making CNNs highly effective for **vision and signal-related tasks**.

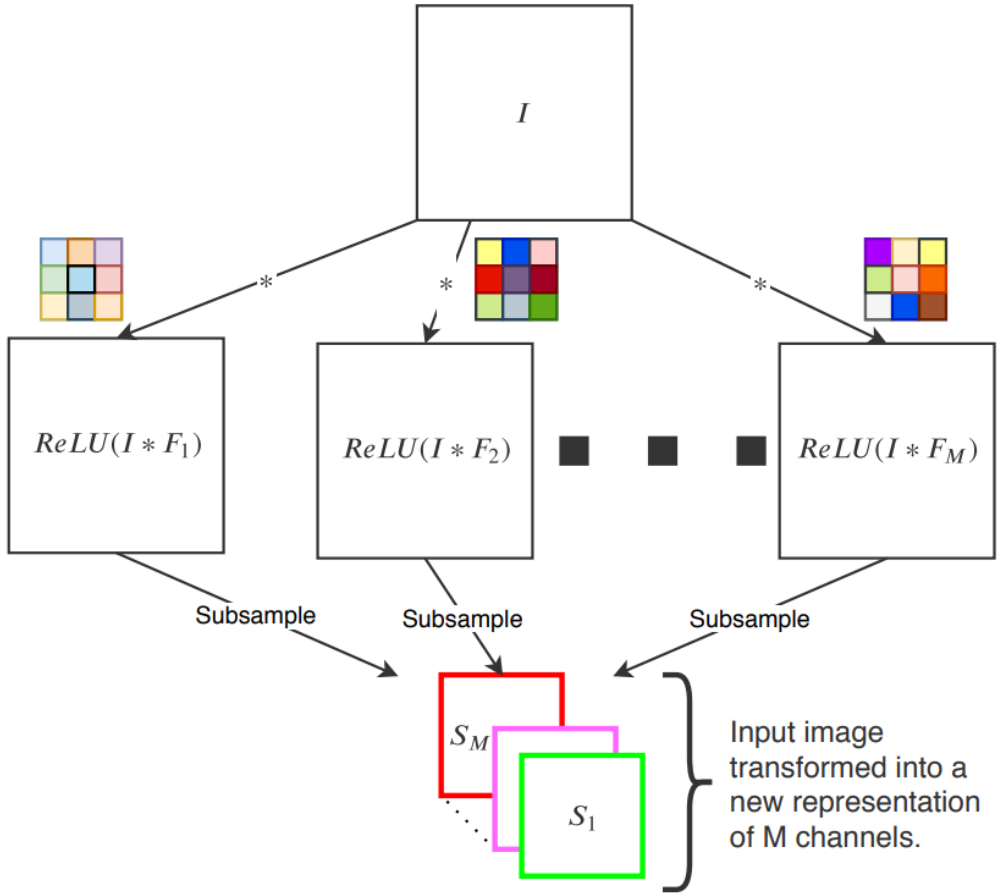
Convolution process



Building blocks of CNN



Building blocks of CNN



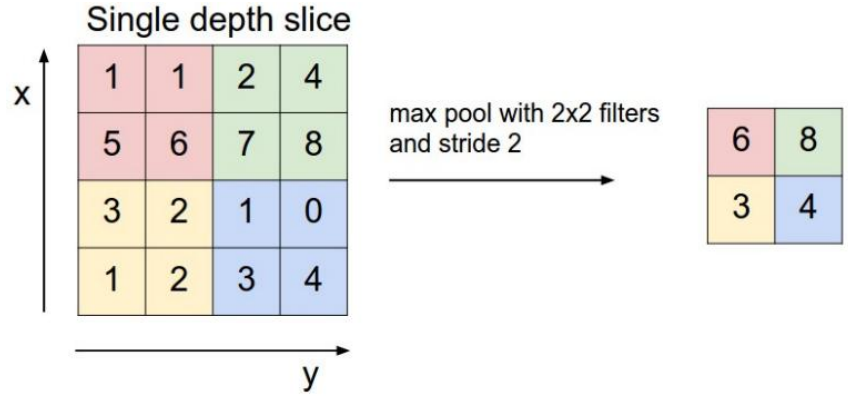
Pooling layer

- It is a component in a CNNs that reduces the spatial size of feature maps while preserving important information.

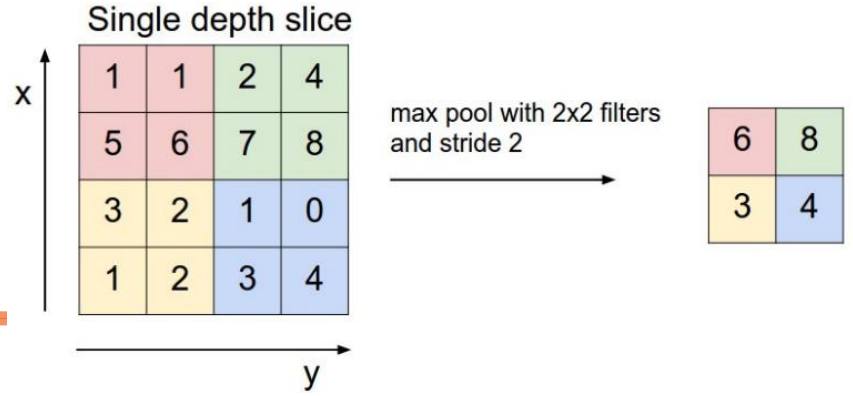
- **How it works?**
  - A small window (e.g., 2×2) slides over the feature map
  - It summarizes values within that window into a single value

- **Resultant Size**

$$\frac{W - F}{S} + 1$$



Pooling layer

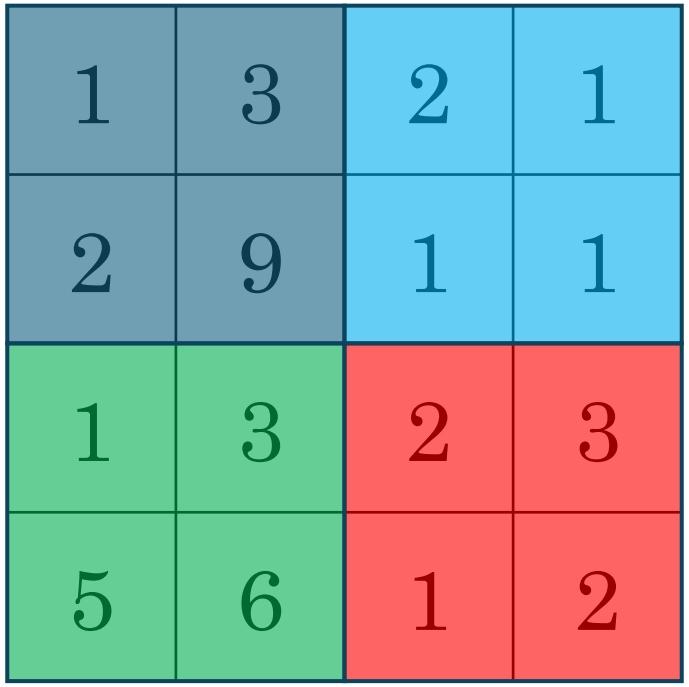


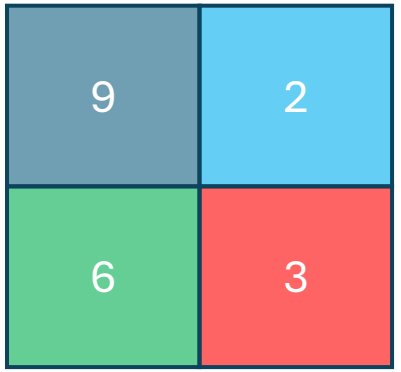
- **Common types**
  - **Max Pooling** → selects the maximum value (most common)
  - **Average Pooling** → computes the average value

- **Purpose:**
  - **Dimensionality reduction** → fewer parameters, faster computation
  - **Noise reduction** → retains dominant features
  - **Translation invariance** → small shifts in input don’t affect output much

Pooling layer

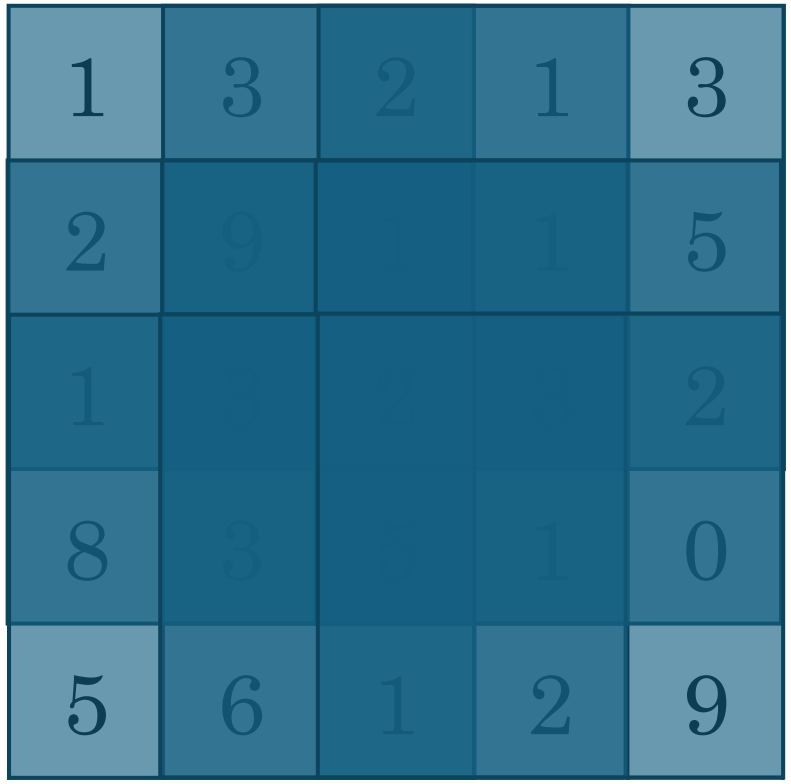
- Max pooling, n=4x4, filter 2x2, stride=2





Pooling layer

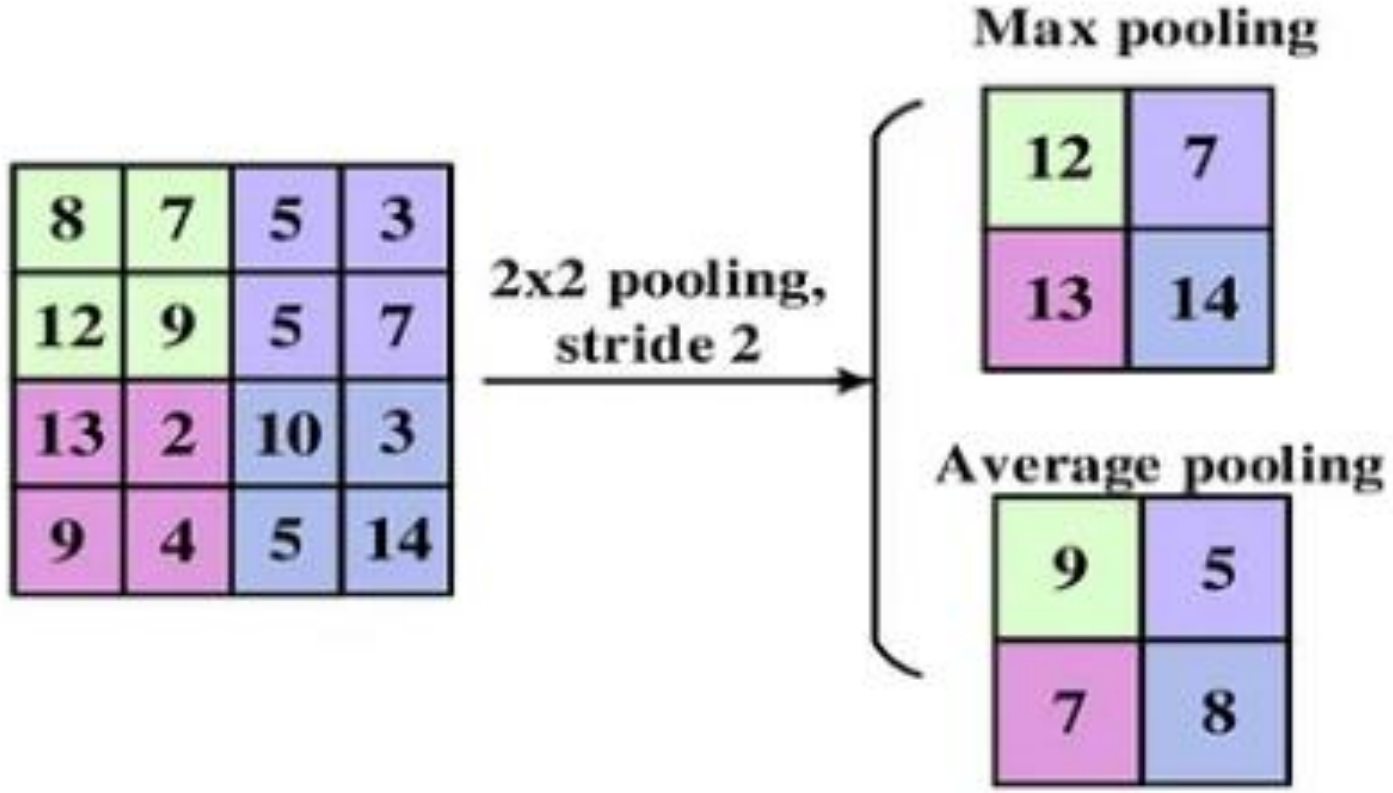
- Max pooling, n=5x5, filter 3x3, stride=1



9	9	5
9	9	5
8	6	9

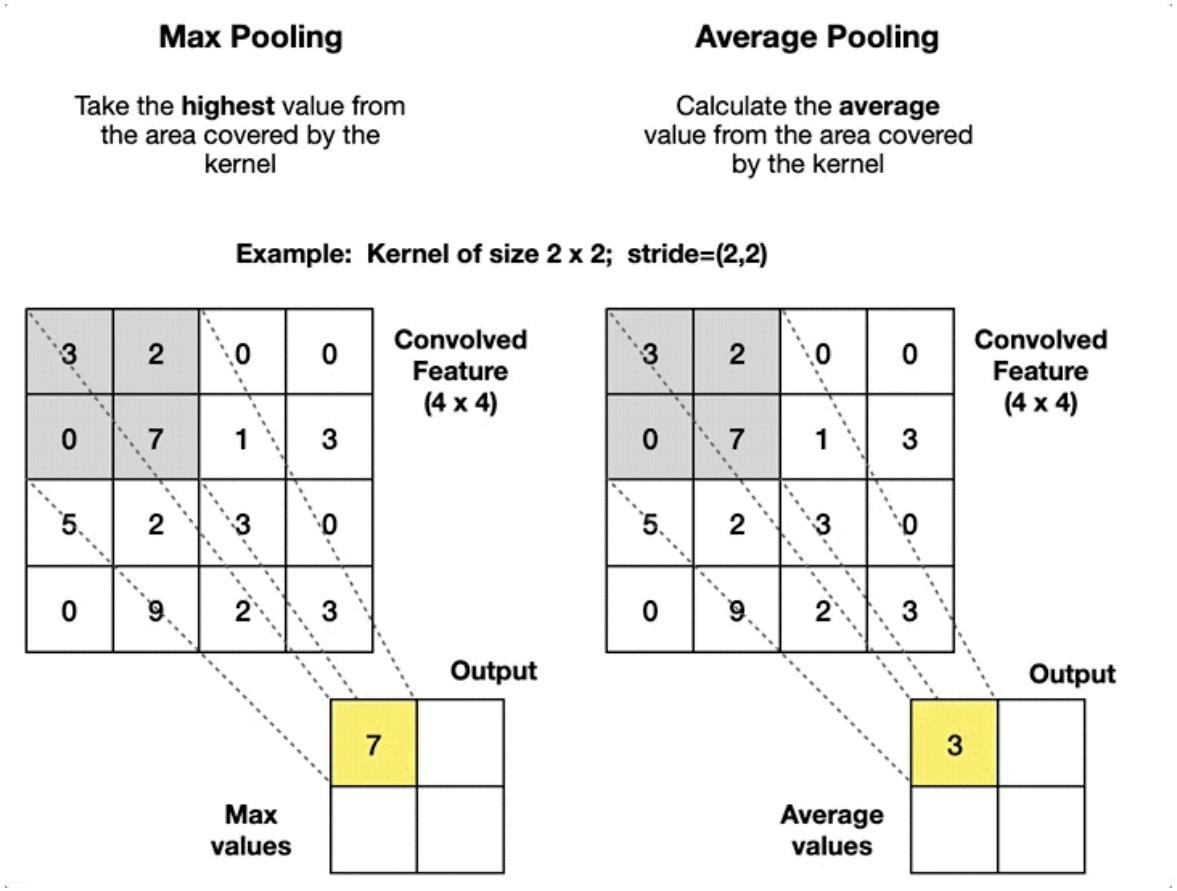
Pooling layer

- Max pooling vs average pooling



Pooling layer

- Max pooling vs average pooling



Fully connected Layer (FC)

- It is also called a **dense layer**.
- It is where each neuron is connected to every neuron in the previous layer (standard **artificial neural network layer**.).
- The **feature map** is **flattened into a vector** before being fed into the dense layer.

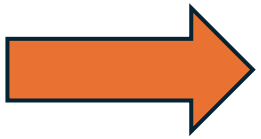
Flattening

- Flattening is the process of converting multi-dimensional data (e.g., a 2D or 3D feature map) into a 1D vector.
- Convolution and pooling layers produce feature maps (matrices)
- Fully connected (dense) layers require vector input
- Flattening bridges this gap

Flattening

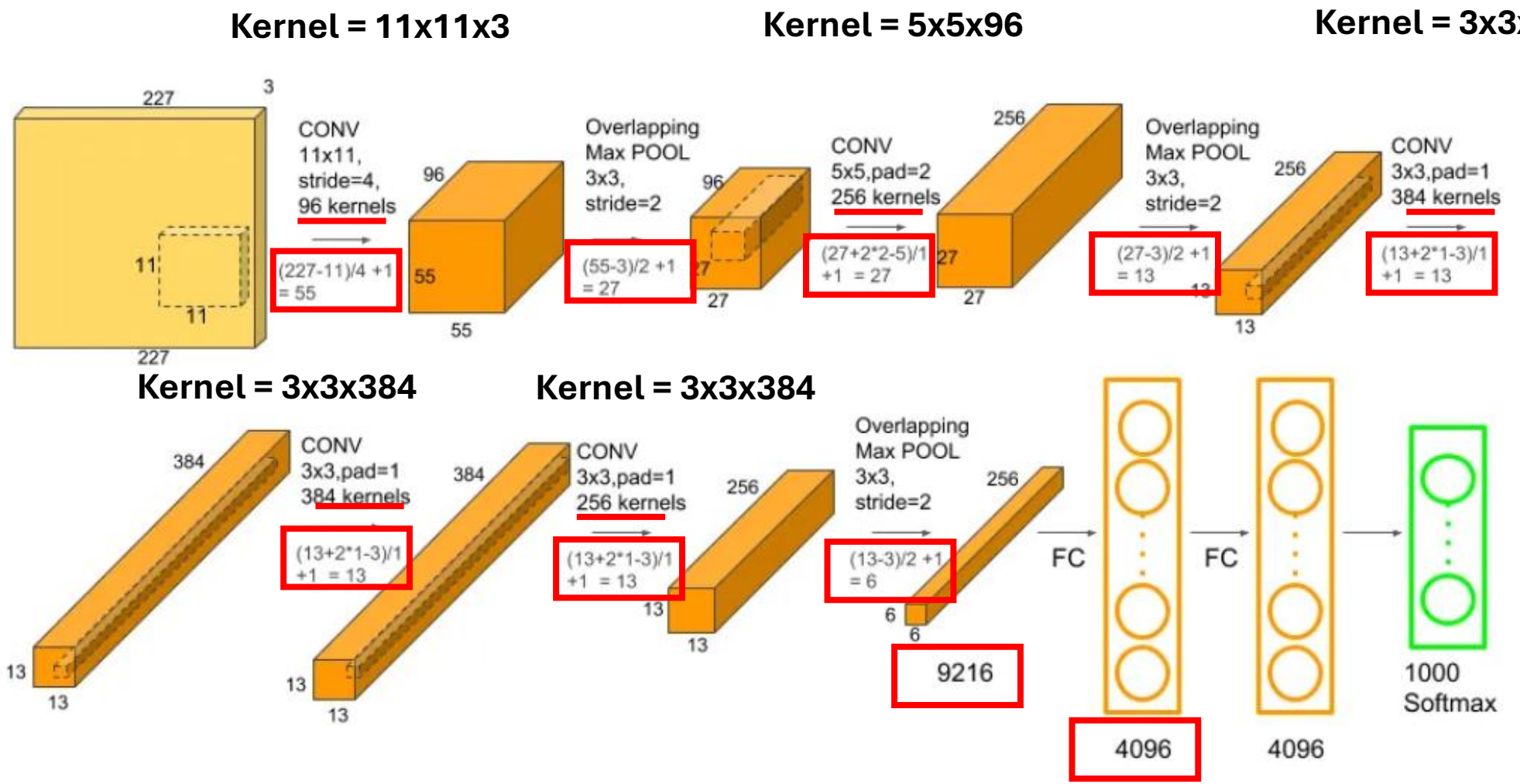
- A feature map of size 3 × 3:

1	2	3
4	5	6
7	8	9

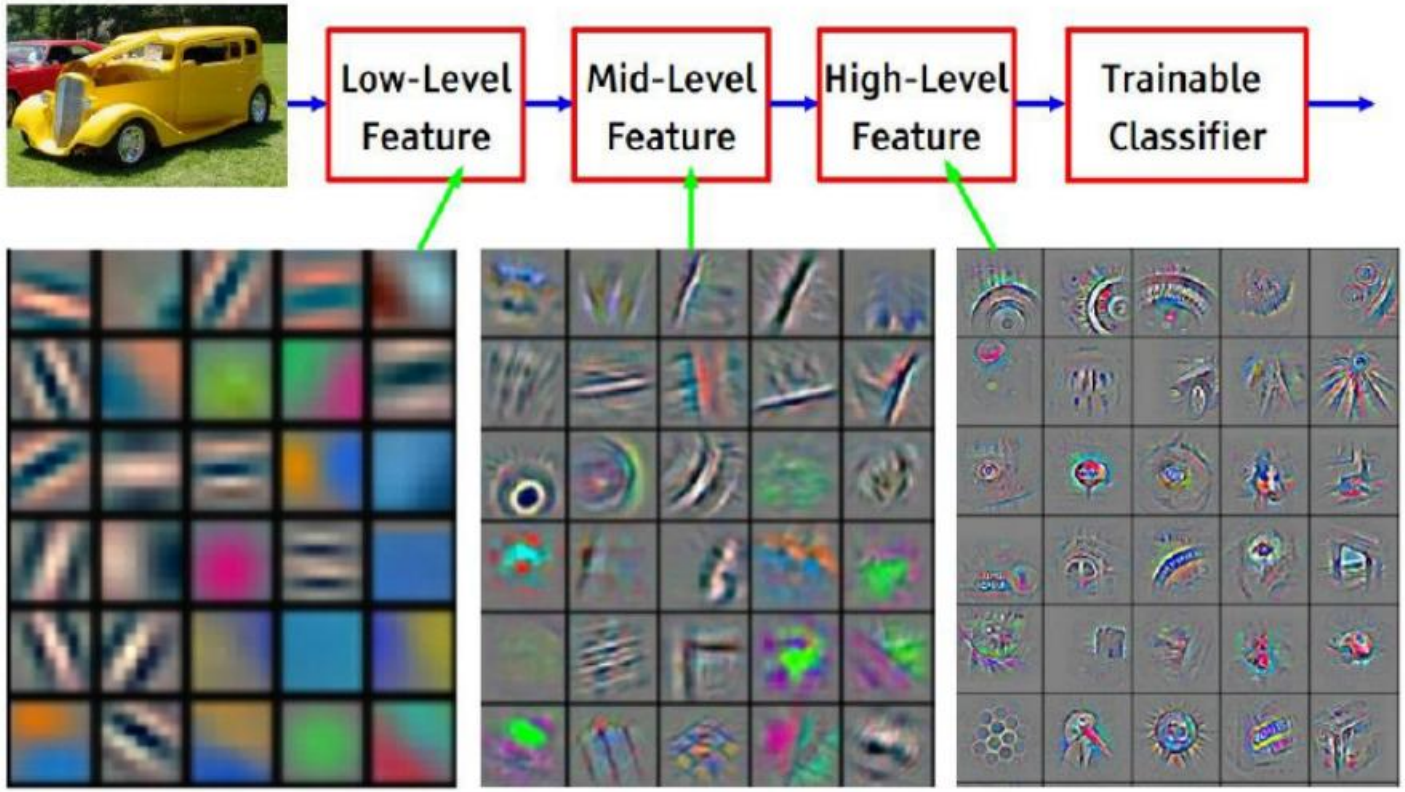


1
2
3
4
5
6
7
8
9

CNN

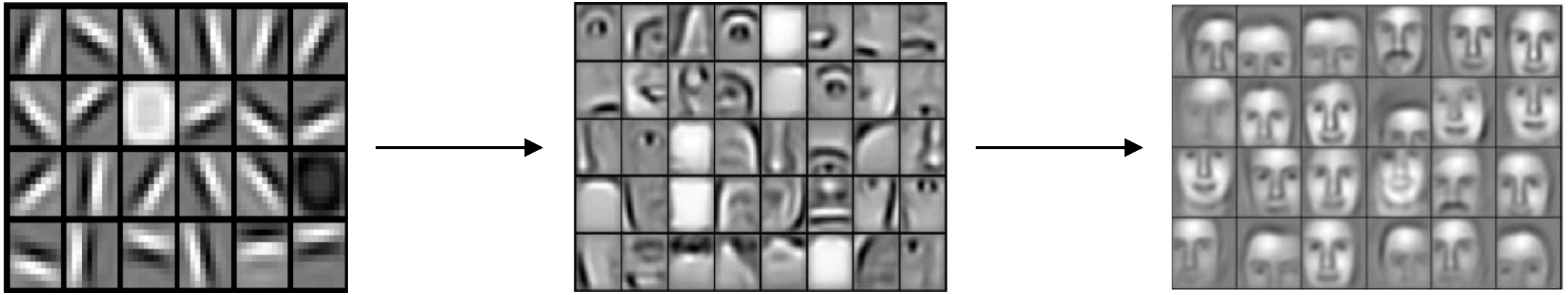

Kernel = 11x11x3
Kernel = 5x5x96
Kernel = 3x3x256
227
3
11
11
227
CONV
11x11,
stride=4,
96 kernels
(227-11)/4 +1
= 55
96
55
55
Overlapping
Max POOL
3x3,
stride=2
(55-3)/2 +1
= 27
96
27
27
CONV
5x5,pad=2
256 kernels
(27+2*2-5)/1
+1 = 27
256
27
27
Overlapping
Max POOL
3x3,
stride=2
(27-3)/2 +1
= 13
256
13
13
CONV
3x3,pad=1
384 kernels
(13+2*1-3)/1
+1 = 13
Kernel = 3x3x384
Kernel = 3x3x384
384
13
13
CONV
3x3,pad=1
384 kernels
(13+2*1-3)/1
+1 = 13
384
13
13
CONV
3x3,pad=1
256 kernels
(13+2*1-3)/1
+1 = 13
256
13
13
Overlapping
Max POOL
3x3,
stride=2
(13-3)/2 +1
= 6
256
6
6
9216
FC
4096
FC
4096
1000
Softmax

Intermediate representation

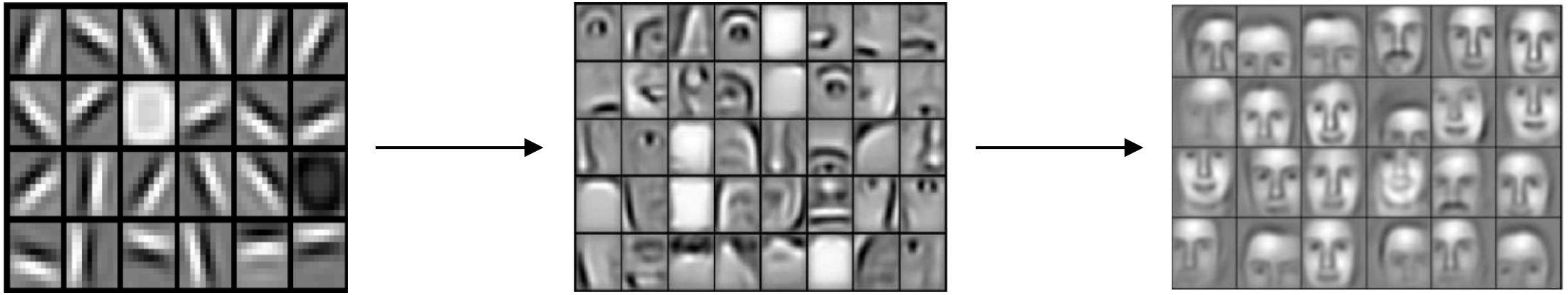


Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Intermediate representation



Intermediate representation



Trainable Parameters in CNN

- **(A) Convolutional Layer**
  - Parameters $= (F_h \times F_w \times C_{in}) \times C_{out} + C_{out}$
  - Where:
  - F_h, F_w =filter height & width
  - C_{in} =number of input channels
  - C_{out} =number of filters (output channels)
  - $+C_{out}$ =bias terms

Input: 32×32×3 (RGB image)
Filters: 10 filters of size 3×3
$(3 \times 3 \times 3) \times 10 + 10 = 270 + 10 = 280$
Total parameters = 280

Trainable Parameters in CNN

- **(B) Fully Connected (Dense) Layer**
  - Parameters $= (\textit{input units} \times \textit{output units}) + \textit{output units}$

Input = 100 neurons
Output = 50 neurons
$100 \times 50 + 50 = 5050$

Trainable Parameters in CNN

- **(C) Pooling Layer**
  - **No trainable parameters**
  - Only reduces spatial size

Example

- **Input:**
  - $32 \times 32 \times 3$
- **Conv Layer:**
  - 3×3 filter, stride = 1, padding = 1
  - 16 filters
  - **Output:**
    - Size = 32 × 32
    - Channels = 16
    - Output = **32 × 32 × 16**
- **Parameters:**
  - $(3 \times 3 \times 3) \times 16 + 16 = 448$

Example

- **Max Pooling (2×2, stride 2)**
- Output:
  16 × 16 × 16
  (No parameters)

Example

- **Flatten**
  - $16 \times 16 \times 16 = 4096$
- **Fully Connected (4096 → 100)**
- Parameters:
  - $4096 \times 100 + 100 = 409700$

CNN & Trainable parameters

Sr.	Operation	Kernel	Activation Shape	Activation Size	# Parameter
1.	Input Layer: 32x32x3				
2.	Conv1(f=5, s=1) 8 filters				
3.	Pool1				
4.	Conv2(f=5, s=1) 16 filters				
5.	Pool2				
6.	FC3 Neurons = 120				
7.	FC4 Neurons = 84				
8.	Softmax Neurons = 10				